POLITIQUE DE GESTION DES DONNÉES SENSIBLES

Projet RCM_LYS

Mars 2026 — Document DevSecOps

1. Politique de gestion des secrets

1.1 Principes fondamentaux

Les secrets (mots de passe, clés API, tokens, credentials de base de données) ne doivent JAMAIS apparaître dans le code source, les fichiers de configuration commités, ni dans l'historique Git. Un secret commité par accident reste accessible dans l'historique Git même après suppression dans un commit ultérieur.

1.2 Fichier .env

Toutes les données sensibles doivent être stockées dans un fichier .env à la racine du projet. Ce fichier ne doit JAMAIS être commité sur le repository.

Règles obligatoires :

- Le fichier .env est listé dans le .gitignore.
- Un fichier .env.example est commité avec les noms des variables SANS les valeurs, servant de modèle pour l'équipe.
- Chaque développeur crée son propre .env en local à partir du .env.example.
- Les fichiers .env ne sont jamais partagés par email, Slack ou tout autre canal non sécurisé.

Exemple de .env.example (commité) :

DB_HOST= DB_USER= DB_PASSWORD= DB_NAME= API_KEY= JWT_SECRET= PORT=3000

Exemple de .env (NON commité) :

DB_HOST=localhost DB_USER=renomatch_user DB_PASSWORD=MonMotDePasse123! DB_NAME=renomatch_db API_KEY=sk-abc123def456 JWT_SECRET=super_secret_jwt_key PORT=3000

1.3 Variables d'environnement dans le code

Le code doit toujours référencer les secrets via process.env, jamais en dur :

INTERDIT	CORRECT
const password = 'MonMotDePasse123!';	const password = process.env.DB_PASSWORD;
const apiKey = 'sk-abc123def456';	const apiKey = process.env.API_KEY;
const jwtSecret = 'super_secret';	const jwtSecret = process.env.JWT_SECRET;

1.4 Secrets dans la CI/CD (GitHub Actions)

Les secrets utilisés dans les workflows GitHub Actions doivent être stockés dans les GitHub Secrets (Settings → Secrets and variables → Actions).

- Jamais de secrets en clair dans les fichiers YAML.
- Référencement via ${{ secrets.NOM_DU_SECRET }}.
- Seuls les Admin peuvent créer et modifier les secrets.
- Les secrets sont masqués automatiquement dans les logs des workflows.

1.5 Secrets dans Docker

- Ne JAMAIS utiliser ENV ou ARG pour passer des secrets dans le Dockerfile.
- Ne JAMAIS copier le fichier .env dans l'image Docker.
- Passer les secrets au runtime via docker run -e ou docker-compose env_file.
- En production, utiliser Docker Secrets ou un gestionnaire de secrets (Vault, AWS Secrets Manager).

1.6 Procédure en cas de fuite de secret

Si un secret est détecté dans le code ou l'historique Git (par Gitleaks ou manuellement) :

- 1. Révoquer immédiatement le secret compromis (régénérer la clé API, changer le mot de passe).
- 2. Prévenir le DevSecOps et le chef d'équipe.
- 3. Ne PAS simplement supprimer le secret dans un nouveau commit — il reste dans l'historique.
- 4. Le DevSecOps nettoie l'historique Git si nécessaire (git filter-branch ou BFG Repo Cleaner).
- 5. Documenter l'incident dans un rapport de sécurité.

2. Règles RGPD pour les données utilisateurs

2.1 Principes du RGPD

Le Règlement Général sur la Protection des Données (RGPD) s'applique à toute application qui collecte ou traite des données personnelles de résidents européens. En tant que projet développé en France, RCM_LYS est soumis au RGPD.

Les 6 principes fondamentaux :

- Licéité — Avoir une base légale pour collecter les données (consentement, contrat, intérêt légitime).
- Limitation des finalités — Collecter les données uniquement pour des objectifs précis et déclarés.
- Minimisation — Ne collecter QUE les données strictement nécessaires.
- Exactitude — Garder les données à jour et permettre leur correction.
- Limitation de conservation — Ne pas garder les données plus longtemps que nécessaire.

- Intégrité et confidentialité — Protéger les données contre les accès non autorisés.

2.2 Données personnelles concernées

Voici les catégories de données que l'application pourrait collecter et leurs règles :

Type de donnée	Exemples	Sensibilité	Règle
Identité	Nom, prénom, pseudo	Moyenne	Consentement requis
Contact	Email, téléphone	Moyenne	Consentement requis
Authentification	Mot de passe, token	Haute	Chiffrement obligatoire
Localisation	Adresse, coordonnées GPS	Haute	Consentement explicite
Comportement	Historique, préférences	Moyenne	Information de l'utilisateur
Paiement	CB, IBAN	Très haute	Ne JAMAIS stocker en clair

2.3 Règles d'implémentation pour les développeurs

- Afficher une bannière de consentement cookies/données au premier accès.
- Permettre à l'utilisateur de consulter, modifier et supprimer ses données (droit d'accès, de rectification et d'effacement).
- Permettre l'export des données personnelles au format lisible (droit à la portabilité).
- Logger les accès aux données personnelles pour traçabilité.
- Ne jamais afficher de données sensibles dans les URLs (pas de mot de passe ou token dans les paramètres GET).
- Mettre en place une politique de rétention : supprimer automatiquement les comptes inactifs après une durée définie.

2.4 Mentions légales et politique de confidentialité

L'application doit inclure une page de politique de confidentialité accessible qui décrit : quelles données sont collectées, pourquoi elles sont collectées, combien de temps elles sont conservées, les droits de l'utilisateur, et comment contacter le responsable du traitement.

3. Sécurisation de la base de données MySQL

3.1 Authentification et accès

- Ne JAMAIS utiliser le compte root pour l'application. Créer un utilisateur dédié avec des droits limités.

- Appliquer le principe du moindre privilège : l'utilisateur de l'application ne doit avoir que les droits nécessaires (SELECT, INSERT, UPDATE, DELETE) sur sa base, pas de GRANT, DROP ou CREATE.
- Utiliser des mots de passe forts (minimum 16 caractères, mélange de majuscules, minuscules, chiffres et caractères spéciaux).
- Les credentials MySQL sont stockés dans le fichier .env, jamais dans le code.

Exemple de création d'utilisateur sécurisé :

CREATE USER 'renomatch_app'@'localhost' IDENTIFIED BY 'MotDePasseComplexe!@#2026'; GRANT SELECT, INSERT, UPDATE, DELETE ON renomatch_db.* TO 'renomatch_app'@'localhost'; FLUSH PRIVILEGES;

3.2 Protection contre les injections SQL

L'injection SQL est l'une des failles les plus critiques (OWASP Top 1). Elle permet à un attaquant d'exécuter des requêtes SQL arbitraires sur la base de données.

VULNÉRABLE (injection possible)	SÉCURISÉ (requête paramétrée)
db.query('SELECT * FROM users WHERE email = '" + email + "")	db.query('SELECT * FROM users WHERE email = ?', [email])

- Toujours utiliser des requêtes paramétrées (prepared statements) ou un ORM.
- Ne jamais concaténer des entrées utilisateur dans les requêtes SQL.
- Valider et sanitiser toutes les entrées utilisateur côté serveur.
- Semgrep détecte automatiquement les injections SQL potentielles dans la pipeline CI.

3.3 Chiffrement des données

- Les mots de passe utilisateurs doivent être hashés avec bcrypt (coût minimum : 12 rounds). Ne JAMAIS stocker un mot de passe en clair ou avec un algorithme faible (MD5, SHA1).
- Les données sensibles (email, adresse, téléphone) peuvent être chiffrées au repos avec AES-256.
- La connexion entre l'application et MySQL doit être chiffrée (SSL/TLS) en production.
- Les sauvegardes de la base de données doivent être chiffrées.

3.4 Sauvegardes

- Mettre en place des sauvegardes automatiques quotidiennes.
- Stocker les sauvegardes dans un emplacement séparé du serveur de production.
- Tester régulièrement la restauration des sauvegardes.
- Chiffrer les sauvegardes et limiter l'accès aux personnes autorisées.

3.5 Configuration MySQL sécurisée

- Désactiver l'accès distant au serveur MySQL si l'application est sur le même serveur (bind-address = 127.0.0.1).
- Supprimer la base de données de test (DROP DATABASE test).

- Supprimer les comptes anonymes (DELETE FROM mysql.user WHERE User='').
- Activer les logs d'audit pour traçer les accès.
- Maintenir MySQL à jour avec les derniers correctifs de sécurité.

4. Gestion des tokens et clés API

4.1 Types de tokens

Type	Usage	Durée de vie recommandée
JWT (Access Token)	Authentification des requêtes API	15 minutes à 1 heure
Refresh Token	Renouveler l'access token	7 à 30 jours
Clé API	Authentifier une application tierce	Jusqu'à révocation, rotation régulière
Token CSRF	Protéger les formulaires contre les attaques CSRF	Par session

4.2 Bonnes pratiques JWT

- Utiliser un algorithme de signature fort (HS256 minimum, RS256 recommandé).
- Le JWT_SECRET doit faire au minimum 256 bits (32 caractères aléatoires).
- Le JWT_SECRET est stocké dans le fichier .env, jamais dans le code.
- Définir une durée d'expiration courte pour les access tokens (15 min à 1 heure).
- Implémenter un système de refresh token pour renouveler les access tokens sans demander le mot de passe.
- Stocker les tokens côté client dans des cookies HttpOnly, Secure, SameSite=Strict — jamais dans le localStorage.
- Implémenter une liste de révocation (blacklist) pour invalider les tokens avant leur expiration.

Stockage des tokens côté client :

Méthode	Sécurité	Vulnérabilité	Recommandation
localStorage	Faible	Accessible via XSS	NE PAS UTILISER
sessionStorage	Faible	Accessible via XSS	NE PAS UTILISER
Cookie HttpOnly	Haute	Protégé contre XSS	RECOMMANDÉ

4.3 Gestion des clés API

- Chaque service externe (paiement, email, etc.) doit avoir sa propre clé API.
- Les clés sont stockées dans le fichier .env en local et dans les GitHub Secrets / variables d'environnement de l'hébergeur en production.
- Rotation régulière des clés (tous les 90 jours minimum).

- Révoquer immédiatement toute clé compromise.
- Utiliser des clés différentes pour les environnements de développement, staging et production.
- Limiter les permissions de chaque clé API au strict nécessaire (principe du moindre privilège).

4.4 Protection des endpoints API

- Tous les endpoints sensibles doivent vérifier le token JWT.
- Implémenter un rate limiting pour prévenir les attaques par brute force.
- Valider toutes les entrées utilisateur côté serveur.
- Retourner des messages d'erreur génériques (pas « mot de passe incorrect » mais « identifiants invalides »).
- Logger les tentatives d'authentification échouées.
- Utiliser HTTPS uniquement, jamais HTTP.

5. Récapitulatif des contrôles

Domaine	Contrôle	Outil / Méthode
Secrets dans le code	Détection automatique	Gitleaks (CI)
Secrets dans le code	Détection avant commit	Gitleaks pre-commit hook
Variables d'environnement	Fichier .env + .env.example	Convention d'équipe
Injection SQL	Détection automatique	Semgrep (CI)
Dépendances vulnérables	Scan automatisé	npm audit + Snyk (CI)
Mots de passe utilisateurs	Hashage bcrypt	Review de code
Tokens JWT	Stockage sécurisé + expiration	Review de code
Docker	Pas de secrets dans l'image	Hadolint + Trivy (CI)
RGPD	Consentement + droits utilisateurs	Review fonctionnelle
Base de données	Accès limité + chiffrement	Configuration MySQL

Ce document doit être partagé avec toute l'équipe et mis à jour au fur et à mesure de l'évolution du projet. Chaque membre est responsable du respect de ces règles dans son domaine de compétence.

Document rédigé par le DevSecOps — Projet RCM_LYS — Mars 2026